Description des flux de données - pili-pos SCE 2.0

Ce document parcourt une journée de service complète dans un restaurant à travers le système pili-pos, en montrant quelles données circulent et où à chaque étape. Il complète `DOCUMENT_PROCESSUS_TECHNIQUE_FR.md` (qui décrit l'architecture statique) en suivant les données le long d'un parcours utilisateur.

1. Début de service - Pointage entrée (Événement S)

Action utilisateur. L'opérateur démarre le kiosque. Le compositeur Cage Wayland lance `pili-pos` en plein écran. L'écran de login demande un INSZ. L'opérateur entre 11 chiffres sur le pavé numérique.

Circulation des données.

1. UI (`LoginView.vue`) → IPC `commands::auth::login(insz, pin)`.
2. `pos-db::queries::users::get_user` → ligne SQLite `users` ; le PIN est comparé à son hash bcrypt.
3. Le store Pinia auth enregistre `(insz, name, role)`.
4. L'UI redirige vers `TablesView.vue`. Le store appelle automatiquement `commands::social::work_in()` - un événement fiscal **S** :
   - Clé de doublon FDM construite à partir de `pos_id`, `pos_date_time`, `terminal_id`, `"WORK_IN"`, et du prochain compteur séquentiel (lu sous le verrou de signature, voir *STRATEGIE_COMPTEURS_FR.md* §5).
   - Corps JSON canonique POST sur l'endpoint GraphQL du FDM.
   - Sur succès : le FDM renvoie short signature + compteurs ; ligne ajoutée à `fiscal_events` en une seule transaction ; ligne `social_status` dans `users` basculée à `present`.
   - Le ticket n'est **pas** imprimé pour les événements S.

2. Ouverture d'une table - Commande (Événement P)

Action utilisateur. L'opérateur tape Table 12 ; la grille du menu se charge. Les clients commandent trois plats ; l'opérateur appuie sur "Envoyer en cuisine".

Circulation des données.

1. UI (`OrderView.vue`) appelle `commands::order::add_order_line` une fois par plat. Chaque appel écrit une ligne `order_lines` avec le nom dénormalisé, le label TVA, le prix unitaire (depuis `menu_items`), et les éventuels modificateurs.
2. "Envoyer en cuisine" déclenche `commands::tables::sign_table_order` :
   - Construit un `OrderInput` pour **P** (pro forma) - même forme qu'une vente mais sans paiement.
   - Sérialisé en JSON canonique, stocké dans `pending_signs` avant l'envoi (voir §3, étape 5).
   - Le FDM signe ; on archive l'événement P.
   - `pos-print::kitchen_ticket::format_kitchen_ticket` construit un payload ESC/POS et l'envoie à l'imprimante cuisine (destination TCP distincte de l'imprimante du ticket client principal).

Le compteur fiscal **P** avance de 1. Le compteur **N** reste intact jusqu'à ce que le client paie effectivement.

3. Le client paie - Vente (Événement N)

Action utilisateur. Le client demande l'addition. L'opérateur tape "Payer" sur la table 12. Choisit "Carte". Le terminal carte renvoie OK. L'opérateur confirme.

Circulation des données.

0. **Choix du support du ticket** (si `pos.json::digital_ticket` est configuré) : l'opérateur sélectionne Papier / E-mail / Les deux avant de taper "Payer" ; pour E-mail, une adresse est saisie et validée à l'écran (`delivery::resolve`). Un choix refusé (adresse invalide, terminal non configuré) s'arrête ici, avant tout appel FDM.
1. UI (`CheckoutView.vue`) → `commands::checkout::checkout` avec `{order_id, payment_method: "card", amount_cents, delivery}`.

2. **Vérification du statut FDM.** `FdmClient::status` s'exécute en premier. Si non-opérationnel, la commande retourne `FdmUnreachable` et l'UI affiche une modale bloquante. Aucun compteur n'est consommé, aucune ligne DB n'est écrite.
3. Construit `SaleInput` à partir des lignes de commande, avec le support résolu à l'étape 0 dans `common.ticketMedium`. Ajoute la référence FDM vers l'événement P antérieur de la table 12 pour que le FDM les relie.
4. Sous le **verrou de signature** unique (une seule opération signe à la fois), lecture du prochain numéro N (`peek_next`), sérialisation JSON canonique, stockage de la requête exacte dans `pending_signs` (statut `open`), puis appel `FdmClient::sign_sale`. Si une requête précédente est restée sans réponse, elle est d'abord renvoyée à l'identique (§8).
5. Sur succès FDM, dans une seule transaction SQLite :
   - Avance du compteur N (`fiscal_counters`).
   - Ajout de l'événement fiscal à `fiscal_events`.
   - Marquage de la ligne `orders` `status='closed'`.
   - Écriture de la ligne `payments`.
   - Ligne `pending_signs` passée à `signed` (avec `fiscal_event_id`).
   - Si un support numérique a été demandé : ajout d'une ligne `ticket_deliveries` (`delivery::enqueue_in_tx`) - texte du ticket + URL de vérification déjà figés, adresse masquée conservée en clair pour l'envoi à venir.
6. `pos-print::receipt::sale` construit le ticket client :
   - En-tête (nom du restaurant, adresse, TVA).
   - Détail ligne par ligne.
   - Récapitulatif TVA (A=21%, B=12%, C=6%).
   - Bloc de données de contrôle issu de la réponse FDM.
   - QR Code Model 2, ECC M (version 3, 29×29 modules) à partir de l'URL 38 caractères fournie par le FDM.
7. Trame ESC/POS envoyée par TCP au TM-m30III - sauf pour un ticket "E-mail seul" dont l'envoi réussit dans le budget de 8 secondes (`delivery::deliver`), auquel cas aucun papier ne sort. Le tiroir-caisse n'est PAS ouvert (paiement carte).
8. Si un e-mail a été demandé : `delivery::deliver` tente l'envoi tout de suite via `pos_mail::outbox::send_now` (budget 8 s). Sur succès, la ligne `ticket_deliveries` passe à `sent` et l'adresse en clair est aussitôt effacée. Sinon, elle reste `pending` et le papier de secours part (§10 ci-dessous, remise en file en tâche de fond).

4. Rapport X en cours de service

Action utilisateur. Le gérant tape "Rapport X" dans la vue rapports pour voir les totaux en cours.

Circulation des données.

1. UI → `commands::report::generate_x_report`.
2. Les événements de la période de réservation ouverte sont relus dans `fiscal_events` et agrégés par `pos-fiscal::report_totals::turnover`.
3. Construit `ReportTurnoverXInput` (pas de champ booking-date dans les rapports X selon v1.3).
4. Le FDM signe un événement **R** labellisé `R-X-TURNOVER`. Archivé dans `fiscal_events`.
5. Ticket imprimé pour le gérant. Les compteurs **ne** sont **pas** remis à zéro.

5. Fin de journée - Rapport Z (Événement R, double enveloppe)

Action utilisateur. Le gérant tape "Clôture journée".

Circulation des données.

1. UI → `commands::report::generate_z_report`.
2. La commande émet **deux** événements R de manière atomique :
   - `R-Z-TURNOVER` (`ReportTurnoverZInput`, inclut `report_booking_date`).
   - `R-Z-USER` (`ReportUserZInput`).

3. Sous le verrou de signature (une vente en cours est attendue puis comptée dans le Z) :
    - Lecture de deux numéros R consécutifs.
    - Les deux inputs forment **une seule requête** (M184, `ticket_count = 2` dans `pending_signs`), stockée avant l'envoi.
    - Sur succès, dans une seule transaction SQLite : avance des deux compteurs, ajout des deux lignes `fiscal_events`, insertion d'une ligne `z_reports` référençant l'événement turnover, fermeture de la période d'ouverture.
    - Si la requête n'est pas signée (non envoyée ou refusée), rien n'est archivé et les numéros restent libres. Si la réponse est perdue, la requête reste ouverte et est renvoyée à l'identique : le Z est archivé une seule fois, avec ses effets.
    - Le Z automatique (`close_overdue_day`, tick de 60 s) saute son tour si la signature est occupée et revérifie sous le verrou que la journée est toujours en retard : pas de second Z vide.
4. Sauvegarde de la DB SQLite vers `/var/lib/pili-pos/backups/pos-YYYYMMDD-HHMMSS.db` (heure UTC) via l'API Online Backup.
5. Deux tickets de rapport Z sont imprimés.

6. Fin de service - Pointage sortie (Événement S)

Action utilisateur. L'opérateur tape "Pointer sortie".

Circulation des données.

1. UI → `commands::social::work_out`.
2. Événement **S** construit (flag `work_out=true`). Verrou de signature, JSON canonique, FDM, archivage. `users.social_status` basculé à `absent`.
3. L'UI déconnecte, retour à l'écran de login.

7. Exception - Facture B2B (Événement I + Peppol)

Action utilisateur. Pendant le checkout, l'opérateur active "Générer facture B2B", saisit le n° TVA, le nom, l'adresse et le pays de l'acheteur.

Circulation des données.

1. Le checkout standard se déroule (Événement **N** si vente à emporter, sinon en ligne dans le flux dine-in P/N).
2. Additionnellement, `commands::checkout` invoque `sign_invoice` pour émettre un événement **I** référençant la vente sous-jacente (via `fdmRefs`).
3. Après l'archivage de l'événement fiscal, `commands::invoice::issue_b2b_invoice` est appelé avec les détails du fournisseur + de l'acheteur et les lignes de vente :
    - `pos-peppol::ubl::to_ubl_xml` construit un UBL BIS Billing 3.0.
    - `AccessPoint::submit` envoie l'UBL à l'AP configuré (actuellement `LoggingAccessPoint` ; câblage Storecove suivi dans `DECISION_PEPPOL_AP_FR.md`).
    - Ligne ajoutée à `peppol_deliveries` avec statut `accepted` / `failed` et l'UBL complet conservé pour audit.
4. Le ticket imprimé montre le n° TVA de l'acheteur et la phrase "Une facture a été émise et délivrée via Peppol".

8. Exception - FDM injoignable en cours de service

Action utilisateur. Une coupure réseau brève survient entre deux commandes.

Circulation des données.

1. Le sondage de fond `FdmClient::status` attrape la déconnexion en premier ; le store Pinia FDM bascule à `unhealthy` ; le voyant FDM du bandeau passe au rouge et un voile couvre les écrans de vente.
2. Toute tentative active de "Payer" échoue à la vérification préalable (`FdmUnreachable`) ; l'opérateur voit la modale bloquante.

3. Il n'existe aucune file d'attente hors-ligne : une commande pro forma (Événement P), une ouverture de tiroir (F) ou toute autre signature est refusée tant que le FDM ne répond pas, et rien n'est archivé.
4. Si la coupure survient **après** l'envoi d'une requête (réponse perdue : délai dépassé, connexion coupée, HTTP 500, corps illisible), la requête reste ouverte dans `pending_signs`. Toute opération de signature la renvoie d'abord à l'identique (même corps, même numéro) ; le FDM, s'il l'avait signée, renvoie sa réponse avec `DUPLICATE_REQUEST` et l'événement est archivé une fois, sous son premier numéro, avec ses effets. Si le renvoi est refusé, la ligne passe à `refused` et son numéro est libre. Sans réponse, l'opération en cours est refusée (`FDM_ANSWER_PENDING`) ; la tâche de fond (60 s) renvoie aussi la requête, la caisse se débloque seule.
5. Une fois le FDM à nouveau opérationnel, la modale se referme automatiquement et l'opérateur retente l'opération. Un nouvel essai est une nouvelle requête (nouveau numéro si la précédente n'a pas été signée) : le POS ne réutilise jamais un numéro avec un contenu différent.

9. Tâche de fond - Envoi différé du ticket numérique

Déclencheur. `run_outbox_once` (`src-tauri/src/commands/delivery.rs`) tourne toutes les 30 secondes (`OUTBOX_INTERVAL`, câblé dans `src-tauri/src/main.rs`), tant qu'un `digital_ticket` est configuré.

Circulation des données.

1. Récupération des lignes `ticket_deliveries` restées bloquées à `sending` depuis plus de 90 s (plantage applicatif pendant un envoi) → remises à `pending`.
2. Purge RGPD : toute ligne dont l'adresse en clair a plus de 72 h se fait effacer son `recipient` (`purge_recipients_older_than`) ; une ligne `failed` purgée passe à `purged`.
3. Sélection des lignes `pending` dont `next_attempt_at` est passé (ou jamais fixé), par lots de 20.
4. Pour chacune, réclamation atomique (`pending` → `sending`) puis reconstruction de l'e-mail à partir de `ticket_text` + `verification_url` déjà stockés (même fonction de rendu qu'à l'encaissement, `pos_mail::render::build_email`), et appel à l'API Scaleway TEM.
5. Sur succès : `ticket_deliveries.status = 'sent'`, adresse en clair effacée immédiatement.
6. Sur échec retryable (réseau, 5xx, 429) : `attempts` incrémenté, nouvelle échéance programmée (1 min, 5 min, 15 min, 60 min selon le nombre d'essais), jusqu'à 5 tentatives au total. Au-delà, ou pour une erreur permanente (4xx hors 429), la ligne passe à `failed` sans être réessayée davantage.

Aucune de ces étapes ne touche à `fiscal_events` : le ticket fiscal a déjà été signé et archivé à l'encaissement (§3) ; cette tâche de fond ne fait que rattraper la remise numérique promise.

9 bis. Commande en ligne (plateforme ou webshop)

Seulement si les commandes en ligne sont activées pour l'installation (`COMMANDES_EN_LIGNE_FR.md` §2).

Déclencheur. `pos_hubrise::poller::run_loop` lit les événements HubRise toutes les 30 secondes (`POLL_INTERVAL`), et au démarrage `pos_hubrise::poller::start` reprend les commandes nouvelles et relit celles signées depuis deux jours.

Circulation des données.

1. Réception : ligne `orders` (`source` = canal, `is_fiscal = 0`, ouverte par l'identité système) et ligne `webshop_orders` (`fiscal_state = 'pending'`), lignes rapprochées du menu dans `lines_json`. Statut `received` mis en file pour HubRise (`hubrise_outbox`). Rien n'est envoyé au FDM.
2. Acceptation par un employé (`do_accept_webshop_order`) : réservation `pending` → `signing`, écriture des lignes dans le journal `order_entries`, puis :
   - commande payée : `signSale` (N), paiement `APP` ou `ONLINE`, `costCenter` `PLATFORM` ou `WEBSHOP`, remises en `priceChanges` ; ticket de caisse TVA imprimé ;
   - commande à payer : `signOrder` (P) avec le même `costCenter`.
3. Dans la transaction qui archive l'événement (`fiscal_events`) : `webshop_orders.fiscal_state` = `signed_n` ou `signed_p`, `orders.is_fiscal = 1`, statut `accepted` mis en file pour HubRise.
4. Commande à payer, à la remise : `signSale` (N) avec le même `costCenter`, paiement encaissé à la caisse ; statut `completed`.

5. `pos_hubrise::outbox::run_loop` envoie les statuts en file, dans l'ordre par commande, avec nouvelles tentatives.

Si le FDM échoue à l'étape 2, la commande revient à `pending` et rien n'est mis en file.

10. Récapitulatif de persistance

Action	Tables touchées
Login	`users` (lecture)
Pointage entrée/sortie	`users` (écriture), `fiscal_events` (append S)
Ajout ligne commande	`orders` (insert/update), `order_lines` (insert)
Envoi cuisine	`fiscal_events` (append P)
Paiement	`fiscal_events` (append N), `payments` (insert), `orders` (close), `fiscal_counters` (advance N)
Rapport X	`fiscal_events` (append R-X)
Rapport Z	`fiscal_events` (append R-Z-TURNOVER + R-Z-USER), `z_reports` (insert), fichier de backup DB
Facture B2B	`fiscal_events` (append I), `peppol_deliveries` (insert)
Remboursement	`fiscal_events` (append N M111/M112 référençant l'original)
Ouverture tiroir (F-WAR)	`fiscal_events` (append F), `original_prints` (si l'original manquant est rattrapé)
Mouvement caisse	`fiscal_events` (append F), `payments` (écriture)
Ticket signé avec support numérique	`ticket_deliveries` (append, même tx que `fiscal_events`)
Impression d'un original (vente, remboursement, ou rattrapage)	`original_prints` (une ligne, jamais deux, par événement)
Envoi/réessai/purge du ticket numérique (tâche de fond)	`ticket_deliveries` (écriture des colonnes de statut, jamais `fiscal_events`)

Toute signature s'exécute sous un seul verrou. Une transaction SQLite unique, ouverte après la réponse du FDM, englobe l'avance du compteur, l'archivage de l'aller-retour FDM, la clôture de la ligne `pending_signs` et tout effet de bord sur la commande. Soit tout commit, soit tout rollback ; si le rollback suit une signature réussie, la ligne `pending_signs` reste ouverte et l'opération suivante archive l'événement.